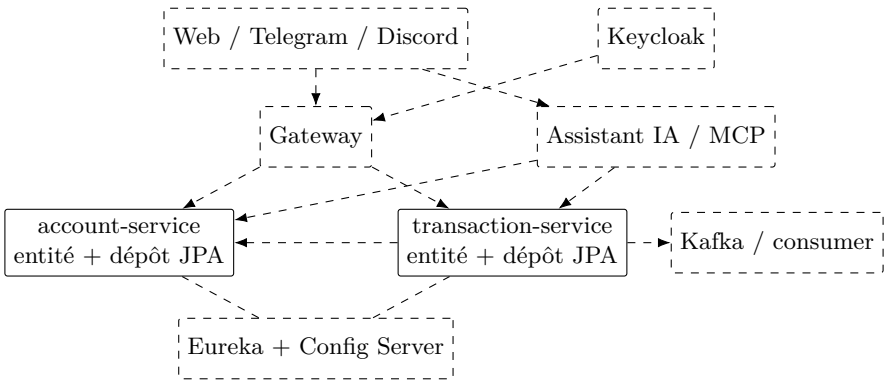
Épreuve technique de sélection

Rapport d'avancement — plateforme bancaire

Youssef Rahioui | 30 octobre 2026

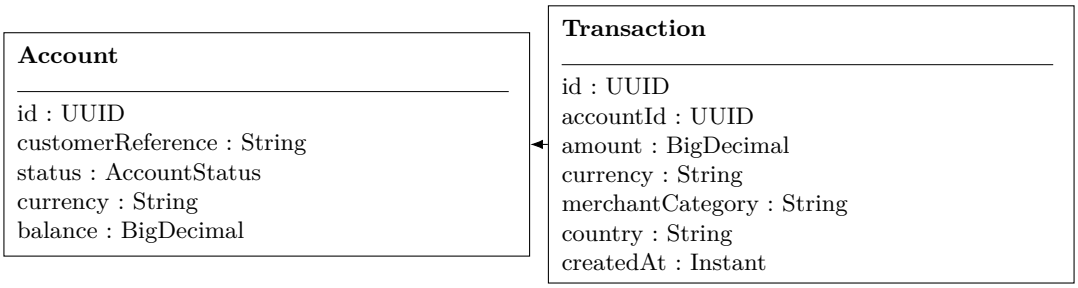
1. Périmètre et architecture (parties A et B)

Le dépôt Maven `bank` contient deux modules : `account-service` et `transaction-service`. Les entités et les dépôts JPA sont présents. Le dessin ci-dessous est une **architecture cible**, et non une représentation des composants déjà opérationnels.



Légende : boîtes pleines = modules partiellement présents ; pointillés = liens/composants envisagés, non réalisés. La liaison entre services prévoit Feign et Resilience4J.

Diagramme de classes simplifié (état du code). Le lien entre les entités est logique, par `accountId`, et non une association JPA.



`AccountStatus` définit `ACTIVE` et `BLOCKED`. Le solde est en `BigDecimal` dans le code, plutôt qu'en `double` comme dans l'énoncé.

2. Réalisation et vérification (parties C à G)

Partie	Constat dans le dépôt
C — métier	Entités et `JpaRepository` présents. `AccountService` et `AccountRestController` existent mais n'exposent aucune opération ; aucun service ni contrôleur de transactions. Pas de GraphQL, MCP, Feign ni Resilience4J. Règles métier, solde et virement non implémentés.
D — technique	Dépendances clientes Config et Eureka dans les deux POM ; pas de serveur Config, de serveur Eureka ou de Gateway. Plafond `bank.transaction.max-amount=1000` absent.
E — agent	Aucun assistant Spring AI ni intégration Telegram/Discord ou test de chatbot.
F — Kafka	Aucun producteur, consumer ou événement `TransactionCreated` ; topic `bank.transactions.created` non utilisé.
G — client	Aucun client web ou mobile présent.

Séparation des couches. Les répertoires `model/` et `repository/` existent dans les deux modules. `service/` et `controller/` n'existent que dans `account-service`. La structure `domain/`, `web/rest/`, `web/graphql/`, `mcp/` de l'énoncé n'est pas encore en place.

Test exécuté. Commande depuis la racine : `mvn test -q`. Le module `account-service` échoue au chargement du contexte ; la suite du réacteur ne s'exécute pas. Extrait de la sortie Maven :

```
APPLICATION FAILED TO START
No spring.config.import property has been defined
Tests run: 1, Failures: 0, Errors: 1, Skipped: 0
Failed to execute goal ...:test on project account-service
```

Cause : le client Spring Cloud Config est déclaré dans le POM, mais `application.properties` ne contient que `spring.application.name`. Ce résultat est un **échec constaté**, pas une validation fonctionnelle. La sortie Surefire complète se trouve dans `account-service/target/surefire-reports/` après exécution.

3. Détection de fraude (partie H)

Aucun fichier `bank_transactions.csv`, notebook ou script Python n'est présent. Aucune métrique de modèle n'est donc disponible. Protocole attendu : séparation stratifiée avant l'ajustement du pipeline (imputation, encodage, normalisation si nécessaire), exclusion des identifiants, apprentissage sur l'entraînement seul, évaluation sur le test par précision et rappel de la fraude, F1, matrice de confusion et average precision. Le seuil ne doit pas être choisi sur le test.

— **Accuracy :** une classe rare peut être ignorée tout en donnant un score global élevé.
— **Compromis :** plus de détections peuvent accroître les alertes légitimes (faux positifs) ; moins d'alertes peuvent laisser passer des fraudes.
— **Seuil plus bas :** rappel potentiellement plus élevé, mais davantage de faux positifs et souvent une précision moindre.
— **Validation temporelle :** elle reproduit une prédiction sur le futur et réduit les fuites liées aux habitudes et à l'évolution des comportements.

4. Exploitation et livraison (parties A et I)

État : aucun Dockerfile, `compose.yaml` ou workflow CI/CD n'est présent dans ce dépôt. Aucun déploiement ou publication d'image n'est démontré. Le dépôt local contient des commits ; sa visibilité GitHub, la feuille Google et le dépôt Classroom ne sont pas vérifiables à partir des fichiers locaux.

Pipeline envisagé : compilation Maven → tests → construction des images Docker → publication au registre → déploiement depuis une version étiquetée.

— **Secrets / configuration :** jetons, mots de passe et clés dans les secrets CI ou un gestionnaire dédié ; paramètres non sensibles (par exemple plafond) dans la configuration versionnée.
— **Santé :** démarrage vérifié avant le trafic ; readiness pour accepter les requêtes ; liveness pour redémarrer un processus bloqué (Actuator peut fournir les sondes).
— **Observabilité :** logs structurés avec identifiant de corrélation, métriques de latence/erreurs et traces distribuées entre gateway, services et Kafka.
— **Rollback :** redéployer l'image précédemment validée et revenir à la configuration correspondante ; privilégier les migrations de données compatibles.

Bilan. Le livrable actuel est un squelette de deux services métier. La première étape pour le rendre exécutable consiste à résoudre la configuration Spring Cloud Config, puis à implémenter et tester les opérations métier.